



Disciplina: Laboratório de Compiladores

Professor: Dr. Ivairton M. Santos

Trabalho Final

No desenvolvimento deste trabalho considere que será necessário empregar os conceitos aprendidos ao longo das disciplinas de Compiladores e Laboratório de Compiladores, em especial aqueles inerentes à elaboração de gramática (para análise sintática), análise *bottom-up* e a utilização das ferramentas FLEX e BISON.

Neste trabalho, implemente um compilador para uma **linguagem C simplificada**, detalhada a seguir, que receba como parâmetro de compilação um arquivo de entrada que deverá conter o código do programa a ser compilado e então que o compilador gere como saída o código de montagem (em linguagem Assembly) correspondente. Caso o código do programa possua erros (seja léxico, sintático, ou semântico) a descrição do erro deverá ser apresentada no terminal (descrevendo o mais detalhadamente o erro, sua linha e posição).

Nesta versão da linguagem C simplificada, o compilador deverá ser capaz de interpretar e reconhecer os seguintes comandos e suas respectivas estruturas:

- Declaração de variáveis:

- As variáveis serão declaradas considerando a estrutura: `<tipo> <identificador>;`

Exemplo:

```
int total ;  
float media ;
```

- As variáveis poderão ser de escopo global e também de escopo local, seja na função principal ou outra função declarada;
 - Será necessário definir e gerenciar as tabelas de símbolos (TS) distintamente:
 - (a) uma TS que corresponde ao escopo global (com declarações fora do bloco de comandos da função principal);
 - (b) as demais TS's de escopo local (declarações feitas no escopo da função principal e das funções que forem declaradas, constituindo uma estrutura de TS encadeada).
 - No uso das variáveis (seja para atribuição ou acesso) será preciso primeiro verificar a TS local (prioridade) e, caso não tenha o cadastro da variável, posteriormente a TS global.
 - O erro semântico deverá ser emitido apenas após o teste com as duas TS;
 - É permitido a declaração de variável local com o mesmo nome de uma variável global. Neste caso, a variável local tem precedência em relação à global.

- Serão reconhecidos os tipos “**int**”, “**float**”, “**char**”, “**bool**” (booleano) e “**string**” (cadeia de caracteres);
- As variáveis serão declaradas considerando a estrutura: `<tipo> <identificador>;`

Exemplo:

```
string nome ;
int total ;
float media ;
```

- Escopo:

- O escopo (bloco de comandos) será delimitado pelos símbolos “{” (abre chaves) e “}” (fecha chaves).

- Comando condicional:

- Será válido apenas o comando condicional “**if**”, bem como sua variação “**if/else**”.

Exemplo 1:

```
if ( <expressão lógica> ) {
    (... comandos ...)
}
```

Exemplo 2:

```
if ( <expressão lógica> ) {
    (... comandos ...)
} else {
    (... comandos ...)
}
```

- Laço de repetição:

- Será válido apenas o comando de laço de repetição “**while**”.

Exemplo:

```
while ( <expressão lógica> ) {
    (... comandos ...)
}
```

- Comandos de leitura/escrita:

- O comando de leitura de dados para uma variável será o “**read**” e deverá obedecer ao padrão: `read ( <identificador_da_variavel> );`

Exemplo:

```
read ( idade ) ;
```

- O comando de escrita “**write**” poderá imprimir uma constante, uma frase (string), ou o dado de uma variável no terminal e deverá obedecer ao padrão:

```
write ( <identificador_da_variavel> );           {para uma variável}
write ( 12345 );                                 {para uma constante}
write ( "Frase de teste" );                       {para uma string}
```

Exemplo:

```
write ( total ) ;
```

- Operadores:

- Os operadores lógico/relacionais são:
 - == (igual)
 - != (diferente)
 - ! (negação)
 - < (menor)
 - <= (menor ou igual)
 - > (maior)
 - >= (maior ou igual)
- Operador de atribuição:
 - = (Exemplo: `contador = 0 ;`)
- Operadores aritméticos:
 - + (soma)
 - - (subtração)
 - * (multiplicação)
 - / (divisão)

- Declaração de Funções:

Para esta versão deverá ser implementado o reconhecimento de funções. Para isso considere as seguintes especificações:

- Toda função deverá ser prototipada no início do código (antes da função principal).
- Toda função deverá ser de um dos tipos primários reconhecidos nesta versão do compilador (não é permitido *void*).
- As funções poderão ter *nenhum*, ou *n* parâmetros. Cada parâmetro deverá ser de um dos tipos primários definidos nesta versão.

Um novo comando deverá ser adicionado ao compilador (ajuste na análise léxica), de modo a permitir o retorno da função, sendo ele o comando: **return**

A gramática deverá ser adequada para o reconhecimento dos seguintes padrões:

- **Prototipação de funções:** a prototipação da função deverá ser uma extensão da regra que reconhece a declaração de variáveis. Lembre-se que o início é o mesmo, mas a prototipação se difere no momento que recebe o caracter ' (')

Uma prototipação deverá obedecer ao padrão abaixo, como nos exemplos:

```
<tipo> <identificador> ( [ <tipo> <identificador> ] * ) ;
```

```
Exemplo1: int soma (int v1, int v2);
```

```
Exemplo2: int fatorial(int v);
```

Os protótipos das funções deverão ser armazenados em uma tabela de símbolos de funções (separada da TS das variáveis). Ela consiste de uma TS ampliada, pois além de registrar o tipo da função e seu identificador, também deve registrar seus os parâmetros (tipo e identificador).

- O bloco de código da função deverá ser especificado entre chaves ('{' e '}')
- A chamada de uma função poderá ocorrer em qualquer bloco de código, obedecendo ao padrão:

```
<identificador> ( [<identificador>]* ) ;
```

```
Exemplo1: resultado = soma (x, 2);
```

Exemplo2: `soma ( 1, 2 ) ;`

Ao reconhecer a chamada da função, o analisador sintático deverá verificar a semântica da chamada, ou seja, se o número de parâmetros informados está de acordo com o protótipo e também os tipos de cada um deles.

- O retorno da função é obrigatório. Além disso, no contexto do retorno (comando), deverá ser verificado se o tipo retornado é compatível com o protótipo da função.
- Finalmente, não é permitido a redefinição de uma função, ou seja, a sua implementação (assinatura) deve ser idêntica ao protótipo.
- A função principal (main) passa a ser obrigatória.

Para a geração de código, vamos assumir algumas simplificações (mas que são próximas do contexto real), a saber:

- A passagem de parâmetros irá ocorrer por meio de uma pilha
- Vamos usar o registrador “ebp” para manipular a pilha de parâmetros
- O retorno da função será armazenado no registrador “eax” (padrão x86)
- Uma sequência padrão para uma função será:
 - *Prólogo*: salva ebp; move esp para ebp; aloca variáveis de escopo local
 - *Corpo*: processa código do bloco/escopo da função
 - *Epílogo*: Armazena valor de retorno em eax, restaura ebp, retorna contexto (instrução ret)
- Na chamada da função, será necessário:
 - Empilhar argumentos – em **ordem inversa**
 - Chamar a função (instrução `call <label>`)
 - Limpar a pilha após o retorno (limpar esp)

Segue exemplo de código fonte e código de montagem correspondente (Assembly):

```
//Protótipo
int soma(int a, int b);
//Implementação
int soma(int a, int b) {
    return a + b;
}

int main() {
    int x;
    x = soma(5, 3); // Chamada
}
```

```
; implemente declaração de variáveis...
section .text
global _start

soma:
    ;Prólogo
    push ebp
    mov ebp, esp
    ;Corpo
    mov eax, [ebp+8] ;carrega 'a'
    add eax, [ebp+12];soma com 'b'
    ;Epílogo
    mov esp, ebp
    pop ebp
    ret

main:
    ;Chamada da função soma(1,2)
    push 2
    push 1
    call soma
    add esp, 8 ;limpa esp
    ;o retorno está na pilha
    push [x]
    ;continua restante do código...
```

Considere o exemplo a seguir de um código completo de um programa, válido e correto para a linguagem C simplificada especificada:

```
float total ;
float media ;
bool impar ;
string empresa ;

float soma(float a, float b);

float soma(float a, float b) {
    return (a + b);
}

int main() {
    string nome ;
    float x ;
    float y ;

    empresa = "UFMT" ;
    write ( "Escreva seu nome:" );
    read ( nome ) ;

    read ( x ) ;
    read ( y ) ;
    total = soma(x,y) ;
    media = total / 2 ;

    impar = False ;
    while ( total > 0 ) {
        if ( impar == False ) {
            impar = True ;
        } else {
            impar = False ;
        }
        total = total - 1 ;
    }

    write ( empresa ) ;
    write ( nome ) ;
    write ( "Sua media eh:" ) ;
    write ( media ) ;
    write ( "Seu numero eh " )
    if ( impar == True ) {
        write ( "impar" ) ;
    } else {
        write ( "par" ) ;
    }
    return 0;
}
```

OBSERVAÇÕES:

- a) Para este trabalho, **deve ser utilizado** como base/referência os códigos exemplos utilizados ao longo do curso.
- b) Apesar da observação anterior, é necessário remover código/declarações/estruturas que não estão sendo empregadas ou que representam contextos estudados que se tornaram obsoletos (versões simplificadas).
- c) Não deve haver restrições de uso, como limite no tamanho do código, expressões aritméticas (ou lógicas) restritivas.
- d) Ao final do processamento do compilador, caso a compilação ocorra com sucesso, deve ser impressa a mensagem informativa e também a tabela de símbolos de variáveis e de funções, construída ao longo da compilação.